

NumPy Tutorial Problems

Linear Algebra (np.linalg) & Random Package (np.random)

Part A — Linear Algebra

1. Create a 3×3 matrix **A** of your choice and compute its **transpose**, **trace**, and **rank**.
2. Given `A = np.array([[4, 2], [1, 3]])`, compute its **determinant** and **inverse**, then verify `A @ np.linalg.inv(A)` gives (approximately) the identity matrix.
3. Solve the system of equations:
$$\begin{aligned} 2x + y - z &= 8 \\ -3x - y + 2z &= -11 \\ -2x + y + 2z &= -3 \end{aligned}$$
using `np.linalg.solve()`, and confirm the result by substituting back into the original equations.
4. Compute the **eigenvalues and eigenvectors** of a symmetric 3×3 matrix. Pick one eigenvalue-eigenvector pair (λ , v) and verify `A @ v ≈ λ · v`.
5. Show that for any square matrix, `trace(A)` equals the **sum of its eigenvalues**, and `det(A)` equals the **product of its eigenvalues**.
6. Compute the **L1, L2, and infinity norms** of a vector using `np.linalg.norm()` with different `ord` values, and explain what each measures.
7. Given a non-square matrix (e.g., shape (4,2)), compute its **pseudo-inverse** using `np.linalg.pinv()` and use it to solve a least-squares problem $Ax \approx b$.
8. Perform **Singular Value Decomposition** (`np.linalg.svd`) on a 4×4 matrix, then reconstruct the original matrix from `U`, `S`, `Vt` and confirm it matches (within floating-point tolerance).
9. Given two matrices `A` (3×2) and `B` (2×4), compute the matrix product `A @ B` and verify the resulting shape follows the $(m,n) @ (n,p) \rightarrow (m,p)$ rule.
10. Use `np.linalg.matrix_rank()` to determine whether a set of 3 vectors (as rows of a matrix) is linearly independent.
11. Compute the **condition number** of a matrix using `np.linalg.cond()` and discuss what a large value implies about solving $Ax = b$ numerically.
12. Given a covariance-like symmetric matrix, verify that all its eigenvalues are non-negative (a property of positive semi-definite matrices).

Part B — Random Package

1. Use `np.random.seed(42)` and generate an array of 10 random floats between 0 and 1 with `np.random.rand()`. Re-run with the same seed and confirm the output is identical.
2. Generate a 5×5 matrix of random integers between 10 and 50 using `np.random.randint()`.
3. Generate 1000 samples from a **standard normal distribution** (`np.random.randn()`), then verify the sample mean ≈ 0 and sample std ≈ 1 .
4. Generate 1000 samples from `np.random.normal(loc=50, scale=5, size=1000)` and check how many fall within one standard deviation of the mean (should be roughly 68%).
5. Simulate rolling a fair six-sided die 10,000 times using `np.random.randint(1, 7, size=10000)`, then compute the empirical frequency of each outcome and compare to the theoretical $1/6$.
6. Use `np.random.choice()` to sample 5 elements **without replacement** from an array of 20 unique items, and separately sample 5 elements **with replacement**, explaining the difference in output.
7. Use `np.random.choice()` with a custom probability distribution (e.g., `p=[0.1, 0.2, 0.7]` over 3 categories) to simulate 1000 weighted draws, then verify the empirical proportions roughly match `p`.
8. Create a dataset as a 2D array of shape (10, 3) (10 samples, 3 features) and use `np.random.shuffle()` to randomly reorder the rows while keeping each row's features together.
9. Split a shuffled dataset of 100 rows into an 80/20 train/test split using random indices generated with `np.random.permutation()`.
10. Generate a random **binomial** sample (`np.random.binomial(n=10, p=0.5, size=1000)`) simulating 1000 repetitions of "10 coin flips," then compute the empirical mean and compare it to the theoretical mean $n \cdot p$.
11. Use `np.random.uniform(low=10, high=20, size=1000)` to generate samples from a uniform distribution, then verify the sample min/max fall within [10, 20] and the mean is close to 15.
12. Compare `np.random.seed()` (legacy global state) with the newer `np.random.default_rng()` generator API — generate the same "10 random integers" task both ways and discuss why the Generator API is now recommended.